Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas.

Integrantes:

Betancourt Enrique;

Navarrete Alejandro &

Salazar Ambar.

Mini-informe ejercicio 04

Compiladores y Lenguajes, IDSD413

GR01CD

Ing. Rodrigo Chancusig MSc.

05 de noviembre del 2026

El ejercicio construye un programa que reconozca si una entrada es un identificador o un numero, teniendo en cuenta que se pueden reconocer entradas como identificador si incian con letras y terminan con números. En la salida del programa, después de ser ejecutado y compilado por Lex y GCC, se reemplazará el texto de entrada por el resultado del análisis, 'IDENTIFICADOR' o 'NUMERO'.

1. Objetivo y funcionamiento

El objetivo es reconocer dos tipos de entrada: numeros e identificadores, entendiendo por números a cualquier cadena de caracteres que solo contenga digitos del 0 al 9, y por indentificadores a cualquier cadena de caracteres que solo contenga letras o letras seguidas por números.

El funcionamiento está dado por las reglas:

```
%%
[0-9]+ printf("NUMERO␣\n");
[a-zA-Z][a-zA-Z0-9]* printf("IDENTIFICADOR␣\n");
%%
```

Listing 1: Reglas de funcionamiento.

Las cuales son las que detectan la naturaleza o estructura de las entradas. Al ser reconocidas, el programa las remplaza por el contenido de aquellos **printf** . Esto se desarrolla por toda la entrada de texto que se inserte de inicio, o de ser el caso recorriendo todo el archivo de texto con el que se elija ejecutar el programa.

Aquellas expresiones regulares que sirven para identificar las cadenas de caracteres se descomponen de la siguiente manera:

Cuadro 1: Significado de los fragmentos de las reglas.

Fragmento	Significado
`[0-9]+`	Uno o más dígitos.
`printf("NUMERO \n");`	Acción: imprime NUMERO cuando se reconoce una secuencia de dígitos.
`[a-zA-Z]`	Una letra minúscula o mayúscula: el identificador debe empezar con una letra.
`[a-zA-Z0-9]*`	Cero o más letras o dígitos después de la primera letra.
`printf(ÏDENTIFICADOR \n");`	Acción: imprime IDENTIFICADOR cuando se reconoce una palabra que empieza con letra.

2. Preparación y compilación

Primero se crea la carpeta de trabajo y se guarda la especificación en `cuatro.l`. El comando `cat cuatro.l` permite revisar el archivo y comprobar que contiene las reglas y la función principal.

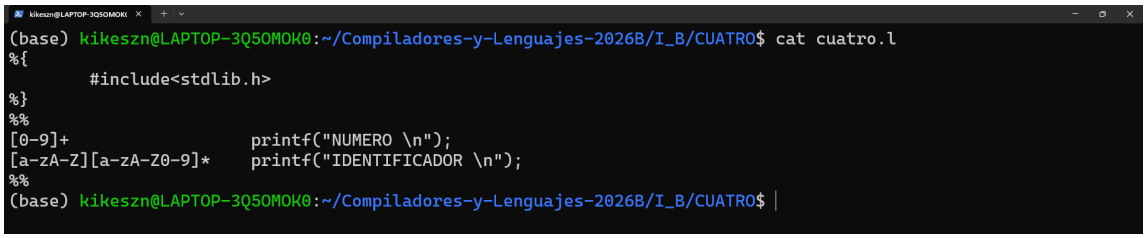


Figura 1: Carpeta y contenido de dos.l

3. Archivo de prueba

Para probar el analizador se crea `entrada.txt` con algunas líneas de texto mediante una entrada multilínea en la terminal. Luego se usa `cat entrada.txt` para confirmar el contenido. La captura permite observar las cadenas que serán procesadas por las reglas del analizador.

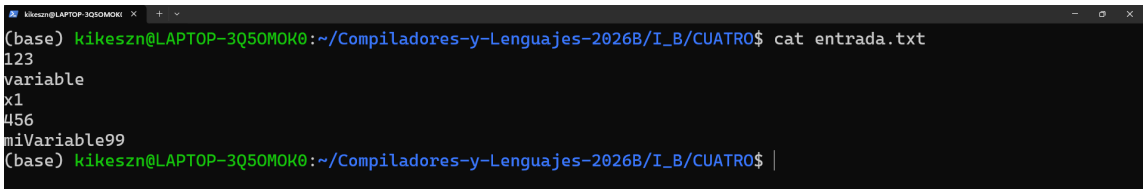


Figura 2: Contenido de entrada.txt

Lex transforma `cuatro.l` y genera `lex.yy.c`. GCC compila ese código generado y, con `-lfl`, enlaza la biblioteca de Flex necesaria para las funciones del analizador. Si la compilación finaliza sin errores, se obtiene `a.out`.

A continuación, la orden `ls` confirma que `entrada.txt` está en la carpeta de trabajo junto con `a.out` y los archivos generados por Lex, como son `cuatro.l` y `lex.yy.c`. Este paso prepara una entrada identificable para la redirección de la siguiente ejecución.

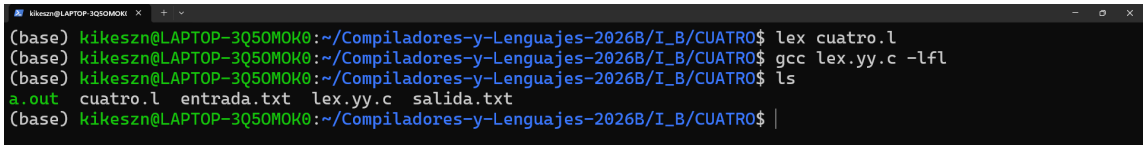


Figura 3: Generación del código C, compilación y archivos resultantes.

4. Ejecución y análisis del resultado

La orden `./a.out <entrada.txt>salida.txt` ejecuta el analizador, toma los datos desde `entrada.txt` y guarda lo que imprime en `salida.txt`. Después, `cat salida.txt` muestra en pantalla el resumen procesado. En la evidencia se reportan 5 reemplazos para las reglas de 'NUMERO' e 'IDENTIFICADOR', y se lo puede comparar y verificar mediante la orden `cat entrada.txt`.

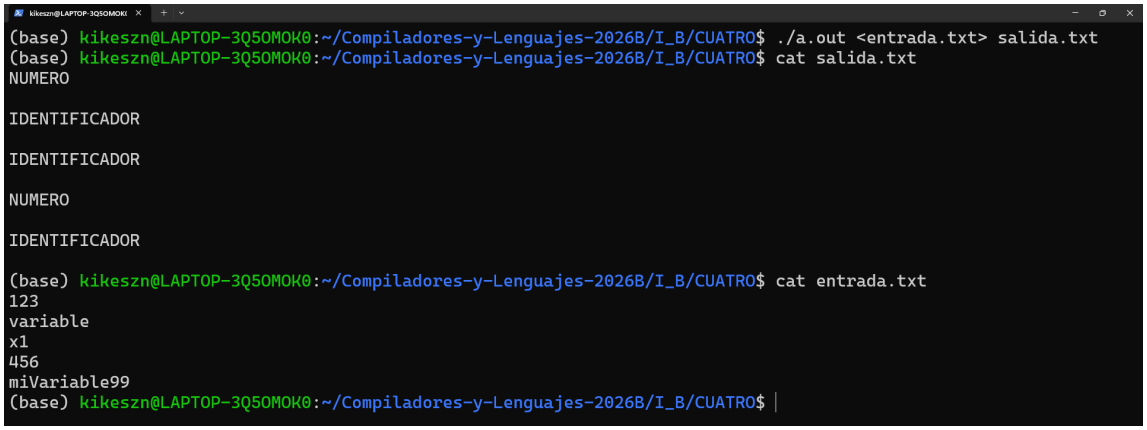


Figura 4: Ejecución con redirección y salida observada.

5. Conclusión

La práctica consolida el flujo para la construcción de un analizador léxico: la definición de patrones, la generación de código en C, su compilación y la validación mediante redirección de archivos. La ejecución nos confirma que para el programa fue fácil reconocer cadenas de caracteres escritas en un texto, sin importar una sintaxis específica, excepto por 'IDENTIFICADOR' que se reconoce como tal cuando existen letras seguidas por números, en el resto de casos el analizador lo resuelve perfectamente. En conclusión, se demuestra que el analizador generado funciona satisfactoriamente si el objetivo es reconocer cadenas de caracteres que se definan en el programa fuente.